

Advanced Git: Guida e Appunti di Studio

1. Come Funziona Git (Le Aree di Lavoro)

Git gestisce il codice attraverso tre aree principali in locale prima di inviarlo al server remoto:

- **Working Directory (o Working Tree):** La cartella di lavoro sul tuo computer dove crei e modifichi fisicamente i file.
- **Staging Area (o Index):** L'area di preparazione. Qui inserisci i file "candidati" che faranno parte del prossimo salvataggio (commit).
- **Local Repository:** Il database locale di Git dove vengono memorizzati i commit (la cronologia ufficiale).

[Working Directory] --(git add)--> [Staging Area] --(git commit)--> [Local Repository]

Comandi Essenziali e Best Practice

- **Aggiungere file alla staging area:**
git add <nome_file>
- **Rimuovere un file dalla staging area (senza cancellarlo dal PC):**
Bash
git restore --staged <nome_file>
- **Creare un commit:**
Bash
git commit -m "Messaggio chiaro e descrittivo"

Best Practice per i Commit: Evitare di committare troppi file non correlati tra loro tutti insieme. Ogni commit dovrebbe avere un **focus unico e specifico** (es. "Risolto bug login" e non "Sistematate un po' di cose in giro"). Questo rende la cronologia (git log) pulita e facile da navigare se devi fare un rollback.

Sincronizzazione con la Repository Remota

Per scambiare codice con il server remoto (GitHub, GitLab, ecc.):

- **Inviare modifiche:** git push
- **Sincronizzare lo stato locale con quello remoto (senza unire i file):**
git fetch
- **Scaricare e unire le modifiche sul tuo codice locale:**
Bash
git pull

2. Gestione dei Branch (Rami)

I branch ti permettono di isolare il tuo lavoro dal flusso principale (main). Puoi sperimentare, scrivere codice incompleto o buggato senza rischiare di rompere la versione stabile utilizzata dal team.

Comandi Principali per i Branch

- **Visualizzare i branch attivi:**
`git branch`
- **Creare un nuovo branch:**
`git branch <nome_branch>`
- **Spostarsi su un branch:**
`git checkout <nome_branch>` o `git switch <nome_branch>`
- **Eliminare un branch (locale):**
`git branch -d <nome_branch>`

Come Effettuare un Merge (Unione)

Quando il lavoro sul tuo branch (es. test) è pronto e stabile, devi portarlo sul branch principale:

1. Spostati sul branch di destinazione:
git checkout main
2. Unisci il branch di lavoro:
git merge test
3. Invia il risultato sulla repository remota:
git push

Gestione dei Conflitti

Se due persone modificano la stessa riga dello stesso file contemporaneamente, Git non saprà quale tenere e genererà un **conflitto**.

- **Come risolverlo:** Usa il tuo IDE (es. VS Code) che evidenzia visivamente le differenze.
- **La regola d'oro:** Non tirare a indovinare. Confrontati sempre con il collega che ha scritto l'altra parte del codice per decidere insieme cosa tenere.

3. I Tag e il Semantic Versioning

I **Tag** sono etichette immutabili collegate a punti specifici della cronologia (solitamente usati per contrassegnare i rilasci delle versioni).

Si dividono in:

- **Lightweight:** Semplici puntatori a un commit (come un branch che non si muove più).
- **Annotated:** Contengono metadati utili (autore, data, messaggio di accompagnamento).

Semantic Versioning (SemVer)

Le versioni seguono lo standard **Major.Minor.Patch** (es. v1.0.2):

Tipo	Quando si incrementa	Esempio
Patch	Piccoli bugfix o refactoring che non aggiungono funzionalità.	v1.0.1 → v1.0.2
Minor	Nuove funzionalità retrocompatibili (non rompono il codice esistente).	v1.0.2 → v1.1.0
Major	Modifiche strutturali che rompono la retrocompatibilità.	v1.1.0 → v2.0.0

Comandi per i Tag

- **Creare un tag annotato:**
git tag -a v0.1.0 -m "Rilascio versione Alpha"
- **Inviare i tag sul server remoto:**
git push --tags
- **Visualizzare la cronologia in modo compatto:**
git log --pretty=oneline

4. Git Flow

Git Flow è un modello di ramificazione (*branching model*) rigido e collaudato che assegna ruoli specifici a branch diversi per gestire al meglio il ciclo di rilascio del software.

I Branch di Git Flow

Branch	Scopo	Origina da	Confluisce in
main	Codice stabile attualmente in produzione. Ogni commit qui è una release taggata.	-	-
develop	Integrazione di tutte le nuove funzionalità pronte. È la base per lo sviluppo.	main	main (tramite release)
feature/	Sviluppo di una specifica nuova funzionalità.	develop	develop
release/	Preparazione al rilascio (test, ultimi bugfix prima della produzione).	develop	main & develop
hotfix/	Correzione immediata di un bug critico presente in produzione (main).	main	main & develop

Comandi Operativi con Git Flow CLI

Per automatizzare i passaggi di Git Flow, puoi usare l'estensione ufficiale integrata in Git.

- **Inizializzare Git Flow in un progetto:**

`git flow init`

Gestione Feature

Inizia una nuova feature

`git flow feature start nome_funzione`

...lavori, fai i commit...

Conclude la feature (fa il merge automatico in develop e cancella il branch locale)

`git flow feature finish nome_funzione`

Gestione Release

Inizia la fase di test e preparazione al rilascio

`git flow release start 0.0.2`

...fai solo bugfix dell'ultimo minuto...

Conclude la release (unisce su main, crea il tag v0.0.2, unisce su develop e cancella il branch)

`git flow release finish 0.0.2`

Gestione Hotfix (Bug in produzione)

Crea un branch direttamente da main per risolvere un bug urgente

`git flow hotfix start risoluzione_bug`

...risolvi il bug e fai il commit...

Conclude l'hotfix (unisce su main con tag, unisce su develop e cancella il branch)

`git flow hotfix finish risoluzione_bug`